

PEFT stands for **Parameter-Efficient Fine-Tuning**. It is the general name for fine-tuning methods that **train only a small number of the model's numbers** instead of all of them.

What each word means

- **Parameter:** one of the numbers inside the model. TinyLlama has 1.1 billion; Qwen3-1.7B has about 1.7 billion.
- **Efficient:** uses much less GPU memory, time and storage.
- **Fine-tuning:** further training a finished model on a specific task.

The problem it solves

	Full fine-tuning	PEFT
Numbers trained	all of them (about 1.1 billion for TinyLlama)	a small set (12.6 million in our TinyLlama run, 1.1%)
GPU memory needed	very high; doesn't fit on a free T4	fits easily
What you save	a whole new model (GBs)	only the small trained part (MBs)
Original model	changed	untouched, so it can be reused for other tasks

LoRA is one PEFT method

PEFT is the **family**. LoRA is the **method we used**, and the most popular one today. Others in the family:

Method	What it trains
LoRA	Small A and B tables next to the model's big tables
QLoRA	LoRA on top of a 4-bit compressed model
Prompt tuning	A few extra "made-up tokens" added to the start of every input; only their numbers are trained
Prefix tuning	Similar, but adds trainable numbers inside every layer
Adapters	Small extra layers inserted between the model's existing layers

All of them do the same basic thing: freeze the original model, add or pick a small set of numbers, and train only those.

1. LoRA (Low-Rank Adaptation)

What is trained: two small tables, A and B, placed next to the model's big tables.

How it works:

1. Freeze the original table (for example $2,048 \times 2,048$).
2. Next to it, add table A ($16 \times 2,048$) and table B ($2,048 \times 16$).
3. The input goes through both the original table and $A \rightarrow B$, and the two results are added.
4. Only A and B are trained.

Numbers: 12.6 million trainable, which is 1.1%. This is what we used.

Good:

- High quality, and the most widely used method today.
- After training, A and B can be **merged** into the original table, so answering is no slower.

Not so good:

- On its own, the frozen model still needs its full memory.

In simple terms ,

1. Start with the normal LLM

Suppose you have a pretrained model:

Llama / Mistral / Gemma / Qwen

It already contains billions of learned parameters.

For example:

LLM = 7 billion parameters

If you want to specialize it for **medical question answering**, the traditional approach would be to fine-tune many/all of those parameters.

That's expensive

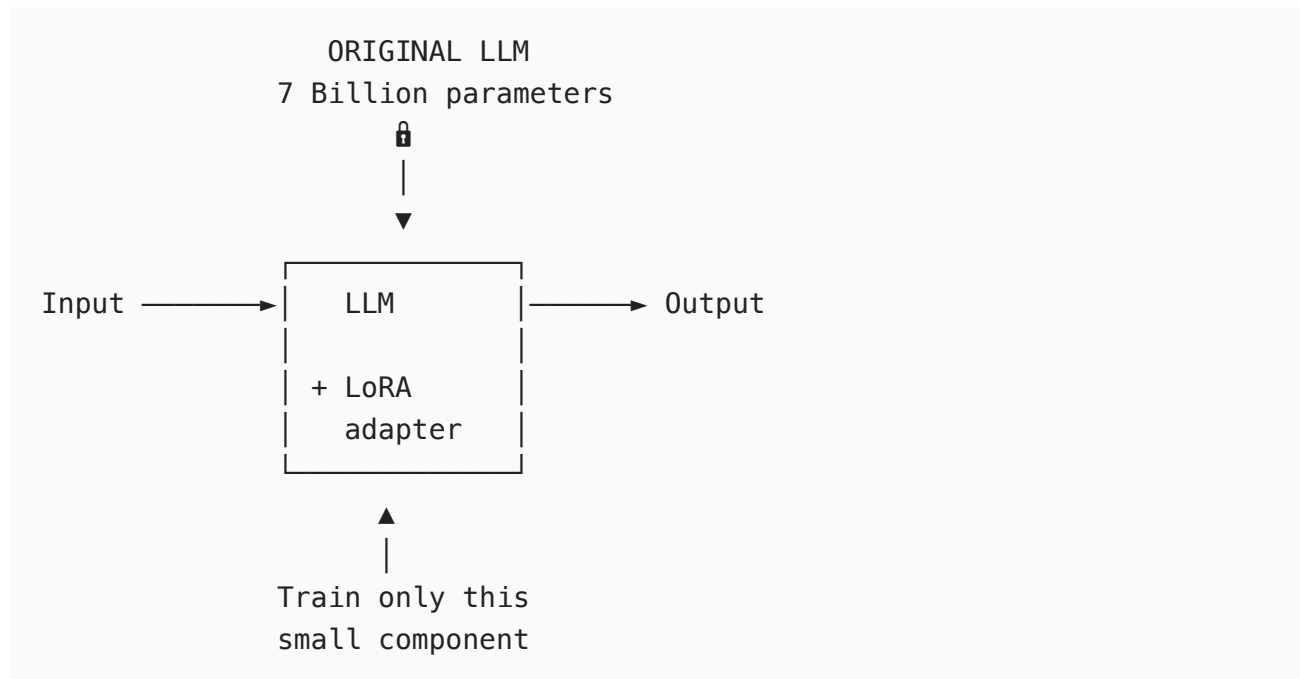
2. What LoRA does

Instead of changing the original 7 billion parameters:

 **Keep the original model frozen**

and attach a **small trainable layer** to certain parts of the model.

Conceptually:



So instead of modifying billions of parameters, you might train only **millions** of additional parameters.

3. The key idea: Low Rank

This is where the name comes from:

LoRA = Low-Rank Adaptation

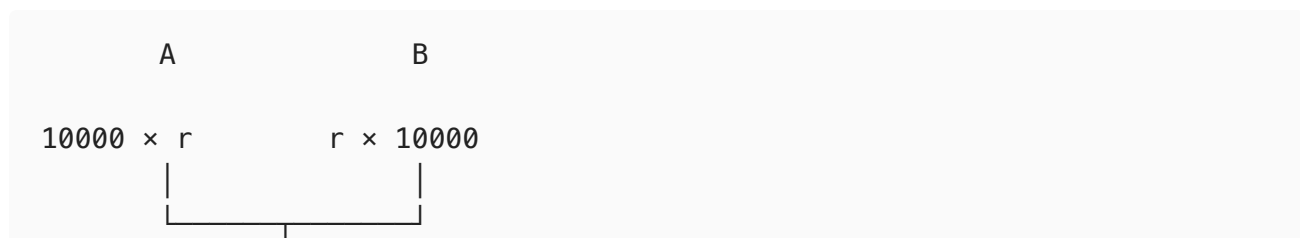
Imagine the model has a huge matrix:

W

10000×10000

That's a **100 million parameter matrix**.

Instead of modifying this entire matrix, LoRA learns a much smaller update using two small matrices:



$$\Delta W = A \times B$$

where r is a small number such as:

$r = 8$
 $r = 16$
 $r = 32$
 $r = 64$

So the model effectively becomes:

New weights = Original weights + LoRA update

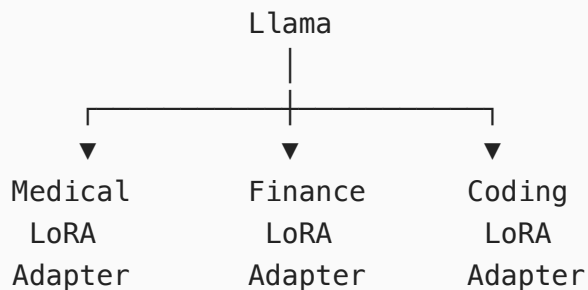
The original weights don't need to change.

4. Why is this useful?

Imagine you have a general-purpose LLM:

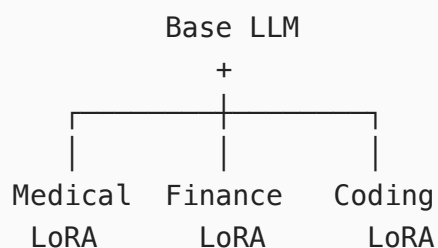
Llama 3

You want three different specialized versions:



You don't need three copies of the entire model.

You can have:



Each LoRA adapter is relatively small.

A **LoRA adapter** is the **saved result of LoRA training**: the trained A and B tables for every layer, plus a small settings file. It's what you get at the end of fine-tuning, separate from the original model.

What's inside it

From your TinyLlama run, the adapter folder held these files:

File	What it contains
<code>adapter_model.safetensors</code> (48 MB)	The trained numbers: all 154 A-and-B pairs (7 tables × 22 layers), 12.6 million numbers in total
<code>adapter_config.json</code> (tiny)	The settings: which base model, rank 16, alpha 32, which 7 table names

The original TinyLlama (2.2 GB) is **not** in it. The adapter only stores the change.

How it is used

1. Load the original base model, TinyLlama, unchanged.
2. Load the adapter on top. The library reads the settings file, finds the 154 tables named there, and attaches the A and B pair next to each one.
3. Now the model answers with the fine-tuned behaviour.

what happens when we train smaller adapter in lora

When you train a smaller LoRA adapter, you are essentially teaching the model **how it should change its behavior**, without changing the original model itself.

Think of it step by step

Suppose the original model has a weight matrix:

```
W = Original model knowledge
```

During normal fine-tuning, we would change `W` :

```
W → W'
```

With LoRA, we **freeze W**:

W 

and learn a small adjustment:

ΔW

So during inference, the model effectively uses:

$$W' = W + \Delta W$$

But ΔW is represented using two small matrices:

$$\Delta W = A \times B$$

So:



What is actually being learned?

Suppose you train a Llama model on **company HR conversations**.

Your training examples might be:

Question:
What is the maternity leave policy?

Answer:
Employees are eligible for ...

During training:

1. The input goes through the frozen LLM.
2. The model produces an answer.
3. The answer is compared with the desired answer.
4. The training calculates the error.
5. **Instead of changing the billions of original weights**, the error is used to update the LoRA matrices A and B.
6. After thousands of examples, those matrices learn a useful adjustment.

So the adapter gradually learns:

"When I see this kind of input, modify the model's behavior in this direction."

Here's the interesting part

Imagine the original model says:

"I don't know the company's preferred format."

After LoRA training, the effective model might produce:

"According to the company's HR policy, employees are eligible for..."

The **base model hasn't been rewritten**.

You've learned a relatively small set of numerical adjustments that steer it toward that behavior.

Why can such a small adapter work?

This is the clever insight behind LoRA.

A huge neural network may have billions of parameters, but for a particular task, you often **don't need to change all of them**.

The required change may lie in a much smaller mathematical space.

So instead of learning:

10 billion new numbers

you might learn:

10 million numbers

or even fewer, depending on the model and LoRA configuration.

One subtle but important point

The adapter isn't literally storing:

```
"maternity leave = 26 weeks"  
"format = bullet points"  
"be concise"
```

It's learning **numerical patterns in the model's weights** that influence how the model responds.

That's why LoRA is fundamentally a **parameter-efficient fine-tuning technique**, not a database.

Let's use a hypothetical LLaMA layer with a **2048 × 2048 weight matrix**, because that's exactly the example you're asking about.

1. Start with the normal model

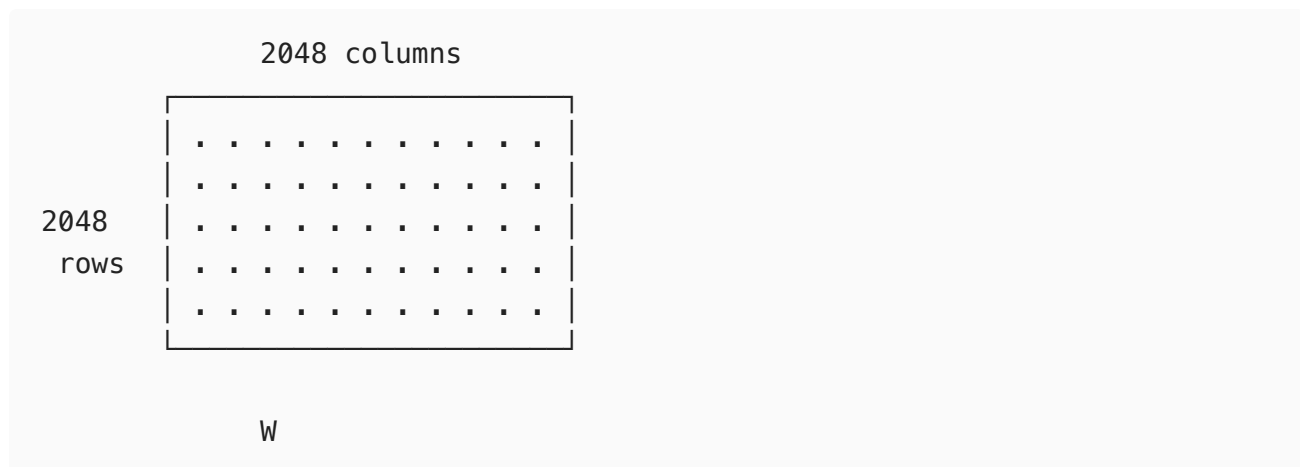
Suppose one layer contains this weight matrix:

$$W = 2048 \times 2048$$

That means:

$$2048 \times 2048 = 4,194,304 \text{ parameters}$$

Imagine it as a giant table:



During normal fine-tuning, we might change all those numbers.

Original W

↓

Fine-tuning

↓

New W

That's expensive because we're training millions/billions of parameters.

2. LoRA says: "Don't touch W "

This is the fundamental idea.

We take:

$$W = 2048 \times 2048$$

and **freeze it**.

W 

Then we create a **small trainable pathway next to it**.

Suppose we choose:

$$\text{rank } r = 8$$

Instead of training:

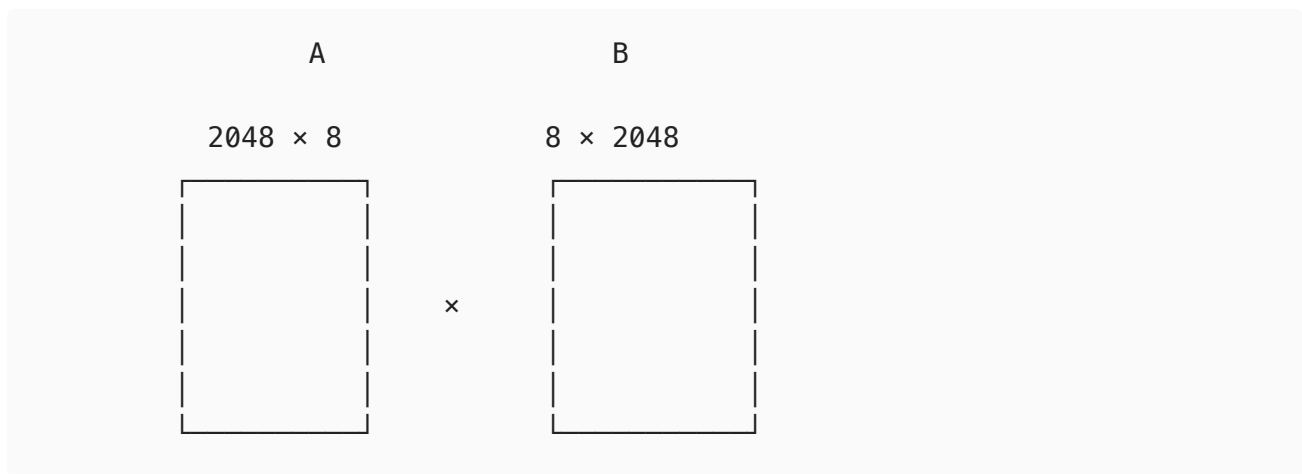
$$2048 \times 2048$$

we create two matrices:

$$A = 2048 \times 8$$

$$B = 8 \times 2048$$

Visually:



Now multiply them:

$$\begin{aligned}
 &A \times B \\
 &(2048 \times 8) \times (8 \times 2048) \\
 &\quad \downarrow \\
 &2048 \times 2048
 \end{aligned}$$

So we get a matrix:

$$\Delta W = A \times B$$

which has the **same shape as W**.

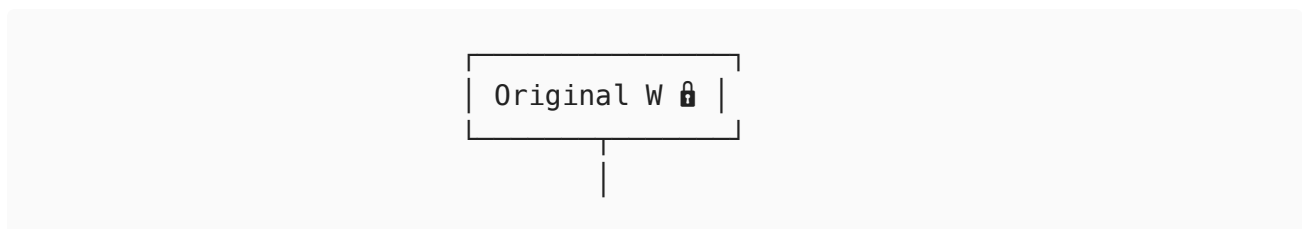
That's the trick.

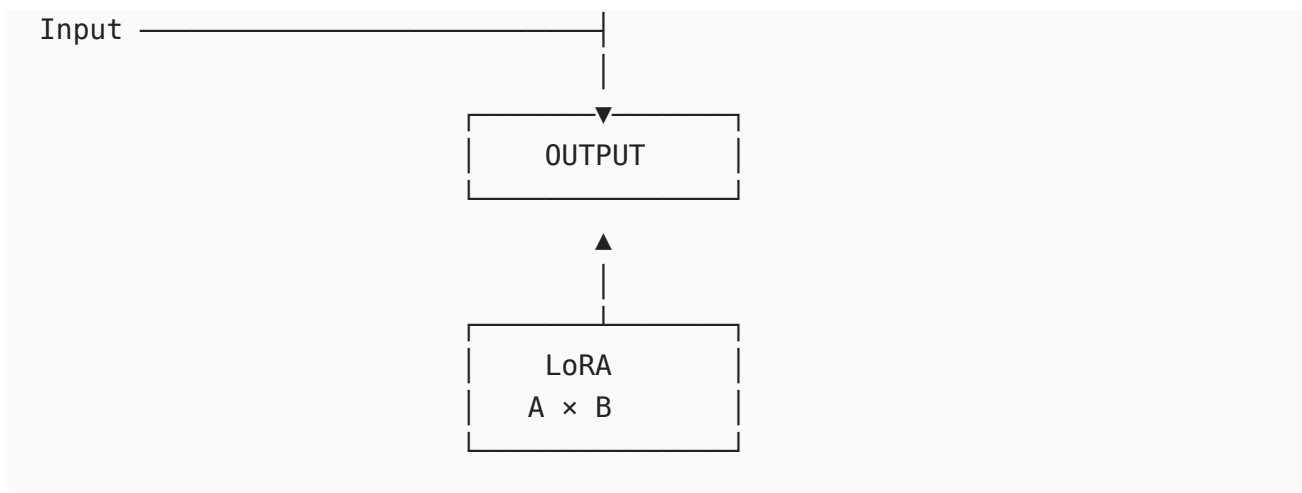
3. So where does LoRA actually go?

This is the part people often explain poorly.

It doesn't replace your original matrix.

You effectively create **two paths**:





Mathematically:

$$\text{Output} = XW + X(AB)$$

or:

$$\text{Output} = X(W + AB)$$

where:

W = original frozen weights
 A, B = trainable LoRA weights

That's LoRA.

4. Let's use ridiculously small numbers

The 2048×2048 example is hard to visualize.

Let's pretend our model has:

$$W = 4 \times 4$$

So:

$$W =$$

[1 2 3 4]

```
[ 5   6   7   8 ]  
[ 9  10  11  12 ]  
[13  14  15  16 ]
```

We freeze it.

W 

Now choose:

rank = 2

LoRA creates:

A = 4 × 2

```
[ a   b ]  
[ c   d ]  
[ e   f ]  
[ g   h ]
```

and:

B = 2 × 4

```
[ i   j   k   l ]  
[ m   n   o   p ]
```

Multiply them:

A × B

and you get:

4 × 4

So:

$\Delta W =$

[... ..]

```
[ ... .. ]  
[ ... .. ]  
[ ... .. ]
```

Notice something important:

ΔW has exactly the same dimensions as W .

That's why we can add them:

```
W +  $\Delta W$ 
```

5. Now comes the training

Suppose you're training LLaMA to become better at **financial analysis**.

You give it:

```
Input:
```

```
"Analyze this company's revenue growth."
```

The original model processes it using:

```
W
```

and produces:

```
Prediction
```

We compare prediction against the desired answer.

That gives us:

```
Loss
```

Now comes the critical difference.

Normal fine-tuning

The optimizer says:

"Let's change W."

```
W ← W - learning_rate × gradient
```

So millions/billions of parameters change.

LoRA

The optimizer says:

"Don't touch W."

W 

Instead:

```
A ← A - learning_rate × gradient_A
```

```
B ← B - learning_rate × gradient_B
```

So only:

A
B

are updated.

Over thousands of training examples:

```
A  
B  
↓  
learn useful transformation
```

6. Why call it LOW RANK?

Now we get to the name.

You have:

$$W = 2048 \times 2048$$

But instead of directly learning another:

$$2048 \times 2048$$

matrix, you're constructing the update through:

$$2048 \times 8$$

$\times$

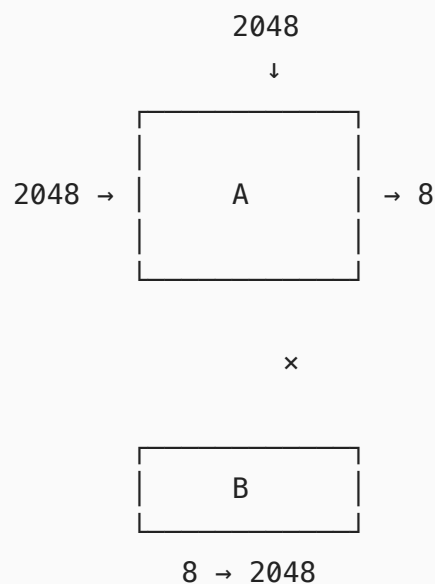
$$8 \times 2048$$

That middle dimension:

8

is the **rank**.

So:



Instead of learning 4.19 million independent parameters:

$$\begin{aligned} &2048 \times 2048 \\ &= 4,194,304 \end{aligned}$$

you learn:

$$\begin{aligned} &2048 \times 8 = 16,384 \\ &+ \\ &8 \times 2048 = 16,384 \\ &= 32,768 \end{aligned}$$

That's dramatically smaller.

7. So what happens to the model after training?

Suppose you trained:

LLaMA
+
Finance LoRA

During training you have:

W 

 $A \leftarrow \text{trained}$
 $B \leftarrow \text{trained}$

At inference:

$$\text{Output} = XW + XAB$$

You can actually **merge** the adapter into the original weights:

$$W_{\text{new}} = W + AB$$

Then you have:

W_{new}

which is again:

2048×2048

The LoRA matrices don't necessarily need to remain separate at inference.

8. But where in LLaMA do we actually put LoRA?

This is another very important question.

You generally don't randomly put LoRA on one arbitrary matrix.

Transformer layers contain matrices such as:

Attention

Q = Query projection

K = Key projection

V = Value projection

O = Output projection

and feed-forward/network projections such as:

Gate

Up

Down

LoRA can be attached to selected linear layers.

A common starting point is:

Q projection

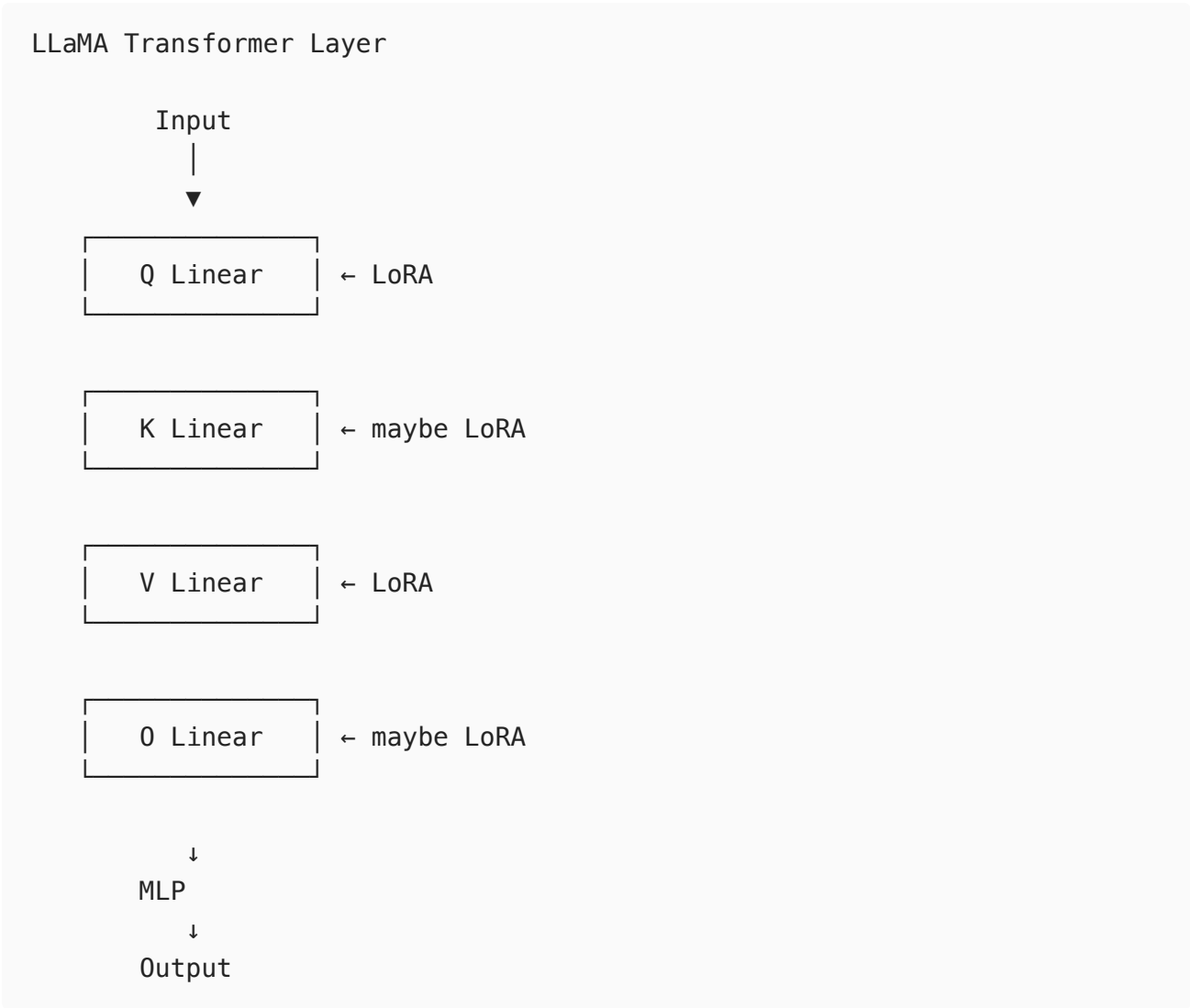
V projection

or sometimes:

Q + K + V + O

and/or the MLP projections.

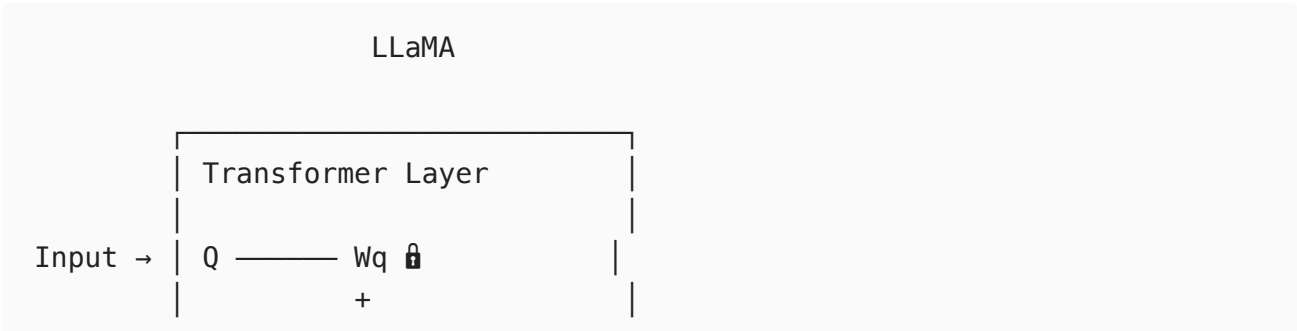
So conceptually:

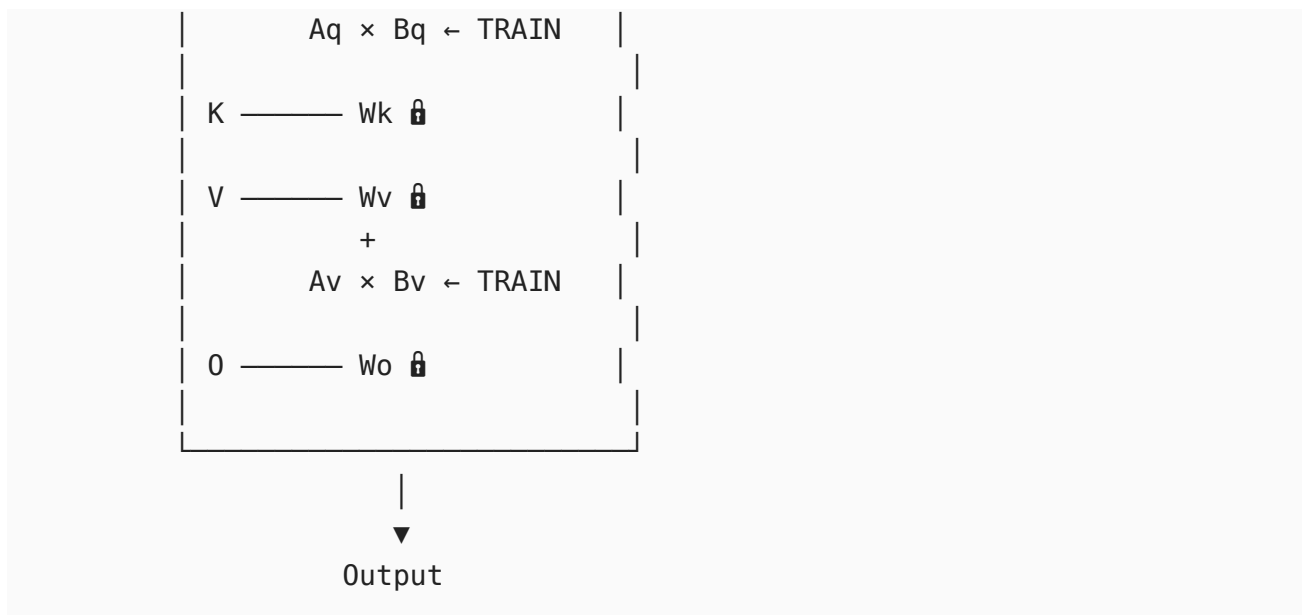


So **LoRA is attached to particular trainable linear layers inside the Transformer.**

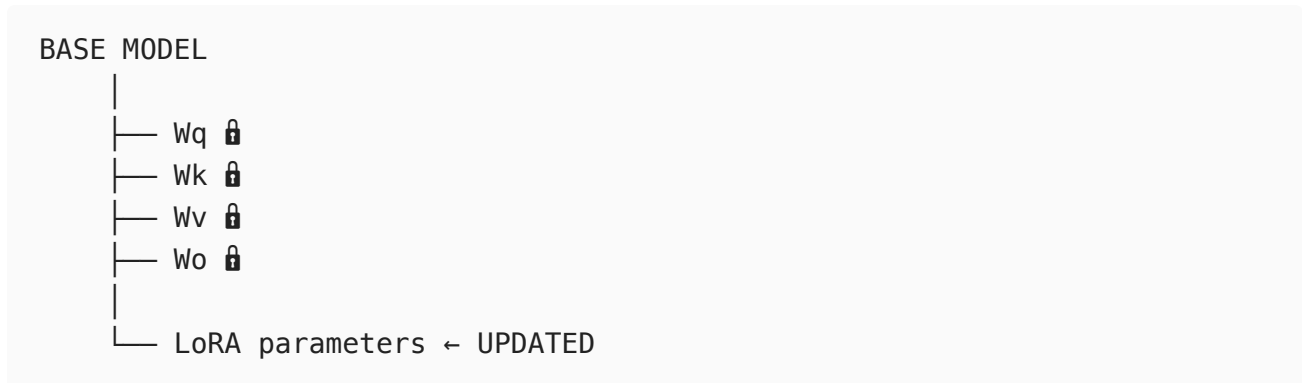
9. The whole picture

Now put everything together.





During training:



So when someone says:

"We fine-tuned LLaMA using LoRA with rank 8."

They're essentially saying:

"We froze the original model and inserted small trainable low-rank matrices into selected Transformer linear layers. During training, only those small matrices were updated."

That is the actual mechanism.

One final distinction worth remembering

LoRA does NOT mean:

"Take the 2048×2048 table and make it smaller."

It means:

"Keep the 2048×2048 table, but learn a low-rank 2048×2048 UPDATE to it by representing that update as 2048×r × r×2048."

That's the conceptual breakthrough.

QLORA

QLoRA becomes very easy once LoRA is clear.

The key idea is:

LoRA reduces the number of parameters we TRAIN. QLoRA additionally reduces the memory needed to STORE the BASE MODEL during training.

First: LoRA

Suppose your LLaMA has a weight matrix:

$$W = 2048 \times 2048$$

LoRA does:

Base W
 🔒 frozen
 +
A × B
 ↑ ↑
trainable

So:

QLoRA

Base LLaMA
 ↓
 ▼
Quantize to 4-bit
 ↓
 ▼
W_{4bit} 🔒
 +
LoRA A × B
 ↑ ↑
trainable

That's basically the whole idea.

Let's use your 2048 × 2048 example

We have:

$$\begin{aligned} 2048 \times 2048 \\ = 4,194,304 \text{ weights} \end{aligned}$$

With FP16:

$$\begin{aligned} 4,194,304 \times 2 \text{ bytes} \\ \approx 8.4 \text{ MB} \end{aligned}$$

With 4-bit:

$$\begin{aligned} 4,194,304 \times 0.5 \text{ bytes} \\ \approx 2.1 \text{ MB} \end{aligned}$$

So the frozen matrix requires **roughly 4× less storage**.

And remember: an actual LLM has **billions** of parameters, not just 4 million.

That's where the savings become huge.

But wait — how can the model calculate with 4-bit weights?

This is where QLoRA gets slightly more interesting.

The base model's weights are **stored quantized**, but during computation the relevant values can be dequantized into a higher-precision representation.

Conceptually:

Storage:

$$\begin{aligned} W &\rightarrow 4\text{-bit} \\ &\downarrow \end{aligned}$$

dequantize
↓
Computation

So don't think:

"The entire neural network is doing all mathematics using crude 4-bit numbers."

Think:

"The frozen weights are stored very compactly, and the system handles the precision needed for computation."

Now the training process

Suppose you give QLoRA:

Question:
Analyze this financial report.

Expected:
Revenue increased because...

The model produces an answer.

Loss is calculated.

Now ask:

Which parameters are updated?

Base model

W_{4bit} 🔒

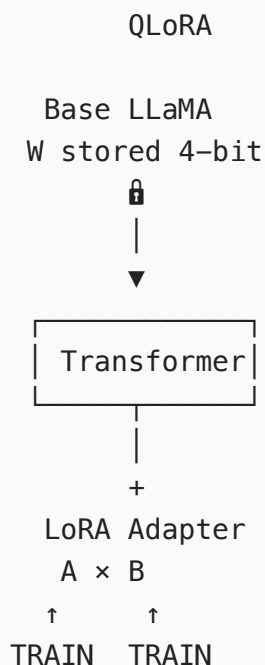
NOT TRAINED

LoRA

A
B

TRAINED 

So:



Why was QLoRA such a big deal?

Because **LoRA already reduced the number of trainable parameters**, but the base model could still consume a huge amount of GPU memory.

Imagine:

70B parameter model

Even if you're training only a tiny LoRA adapter, you still need to **load the 70B base model**.

That's expensive.

QLoRA says:

"The base model is frozen. Why do we need to store all those weights at full precision?"

So:

Full Fine-Tuning

70B model



everything trainable

versus:

LoRA

70B model



← frozen FP16

+

tiny adapter

← trainable

versus:

QLoRA

70B model



← frozen 4-bit

+

tiny adapter

← trainable

LoRA → saves training parameters

QLoRA → saves training parameters + dramatically reduces base-model memory

One more thing: Why "Quantized LoRA"?

Because there are actually **two separate ideas**:

QUANTIZATION

+

LoRA

=

QLoRA

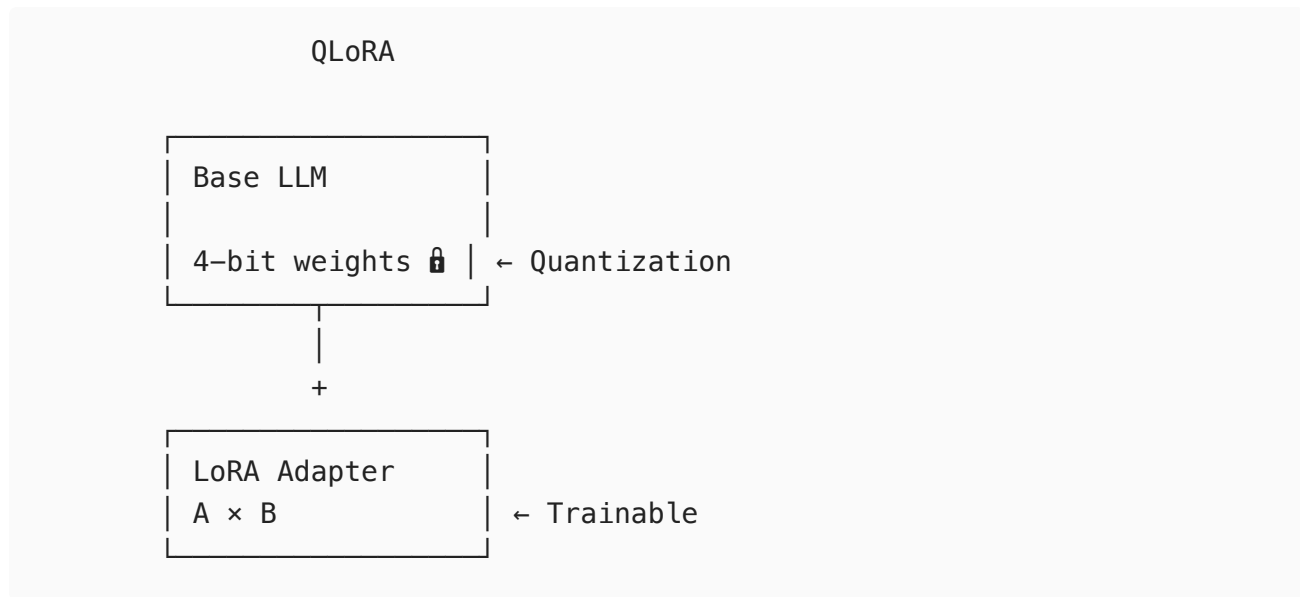
Quantization:

Reduce the precision/storage of the base model.

LoRA:

Train a small low-rank adapter instead of the base model.

Put them together:



And this is the sentence I'd remember:

LoRA says: "Don't train the whole model; train a small adapter."

QLoRA says: "And don't even store the frozen base model at full precision; store it in a highly compressed quantized format."

One subtle point: **QLoRA is not simply "LoRA + any 4-bit quantization."** The original QLoRA technique introduced a specific memory-efficient approach, including **NF4 (NormalFloat 4-bit)**, **double quantization**, and **paged optimizers**. Those are the next pieces to understand if you're going from conceptual knowledge toward actually implementing QLoRA.